



Topic 2: Arrays

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo | McMaster University

Matrix Dimensions

There are several functions to determine the dimensions of a vector or matrix:

- `size` returns the number of rows and columns for a vector or matrix. Both of these values can be captured in the assignment statement
`[row, col] = size(mat)`
- `length` returns the length of the longest dimension of the array.
- `numel` returns the total # of elements in a vector or matrix

⚠: It is very important to generalize your script: do not assume that you know the dimensions of a vector or matrix – use `size` or `numel` to find out!

```
mat = [1 2 3;4 5 6];  
  
[row, col] = size(mat);  
  
numVal = numel(mat);  
  
len = length(mat)
```

Name	Value
mat	[1 2 3;4 5 6]
row	2
col	3
numVal	6
len	3

Matrix Functions

Function	Description	Example <code>matA = [1 2 3; 4 5 6; 7 8 9]</code>	
		Script	Command Window
<code>size(A)</code>	returns <code>[r, c]</code> , the number of rows <code>r</code> and columns <code>c</code> in the matrix <code>A</code>	<code>size(matA)</code>	3 3
<code>length(A)</code>	returns the number of elements along the dimension of the matrix with the largest number of elements	<code>length(matA)</code>	3
<code>numel(A)</code>	Returns the number of elements in <code>A</code>	<code>numel(matA)</code>	9
<code>sum(A, [1 or 2])</code>	Performs same operation as previously defined for vectors, in the direction specified by <code>arg2</code> . - If <code>arg2</code> is set to 1 (default), the operation is performed across the rows of the matrix <code>A</code>. - If <code>arg2</code> is set to 2, the operation is performed across the columns of the matrix <code>A</code>.	<code>sum(matA)</code>	12 15 18
<code>mean(A, [1 or 2])</code>		<code>mean(matA)</code>	
<code>median(A, [1 or 2])</code>		<code>median(matA, 2)</code>	2 5 8
<code>std(A, 0, [1 or 2])</code>		<code>std(matA, 0, 2)</code>	
<code>min(A, [], [1 or 2])</code>		<code>min(matA)</code>	
<code>max(A, [], [1 or 2])</code>		<code>max(matA, [], 2)</code>	3 6 9

A Reminder on Array Operations

MatLab allows you to perform mathematical operations on vectors using ***element-wise*** operators and ***array*** operators.

- Element-wise operators perform operations individually on each element of the vector. These operators include:
 - + addition
 - subtraction
 - . dot operator (couples with other operators such as *, /, ^ to force element-wise behavior)
- Array Operators perform matrix computations, such as matrix multiplication and inversion. These operators include:
 - * multiplication
 - / matrix right division (not used in this class)
 - \ matrix left division (solving systems of linear equations)

⚠ : Element-wise operations can only be completed between vectors of ***equivalent*** shape.

⚠ : Array operations can only be completed between vectors of ***compatible*** shape.

Element-Wise Operations Example

```
m1 = [1 2 3; 4 5 6; 7 8 9];
m2 = [1 1 1; 2 2 2; 1 1 1];
sv = 3;
```

```
mat1 = m1 + m2;
mat2 = m1 .* m2;
mat3 = m1 - m2;
mat4 = m1 ./ m2;
```

```
mat5 = m1 + sv;
mat6 = m1 * sv;
```

$$m1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$m2 = \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & 1 \end{bmatrix}$$

$$sv = \begin{bmatrix} 3 \end{bmatrix}$$

$$mat1 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} + \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 4 \\ 6 & 7 & 8 \\ 8 & 9 & 10 \end{bmatrix}$$

$$mat2 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} .* \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 8 & 10 & 12 \\ 7 & 8 & 9 \end{bmatrix}$$

$$mat3 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} - \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$$

$$mat4 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} ./ \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} & & \\ & & \\ & & \end{bmatrix}$$

$$mat5 = \begin{bmatrix} 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$$

$$mat6 = \begin{bmatrix} 3 & 6 & 9 \\ 12 & 15 & 18 \\ 21 & 24 & 27 \end{bmatrix}$$

: Scalar values perform the same operation on every matrix element.

Linear Algebra

MatLab is a contraction of the phrase *Matrix Laboratory*. This programming language is optimized for matrix operations, and as such, it provides great support for linear algebra. Operations that MatLab simplifies include:

- Matrix Multiplication
- Finding the Transpose of a Matrix
- Finding the Determinant of a Matrix
- Taking the Inverse of a Matrix
- Solving a System of Linear Equations

Linear Algebra: Matrix Multiplication

- In MATLAB, the multiplication operator `*` performs matrix multiplication.
- To multiply a matrix A by a matrix B, the number of columns of A must be the same as the number of rows of B.
 - If the matrix A has dimension of $m \times n$, this means that matrix B must have dimensions of $n \times \text{something}$. In other words, the **inner dimensions** of the matrices must be the same.
 - The resulting matrix C has the same number of rows as A and the same number of columns as B. In other words, the matrix C takes on the **outer dimensions** of the matrices A and B.

⚠: Matrix multiplication is an **array operation**. It does NOT mean multiplying term by term (element by element).

```
>> matA = [1 2 3 4; 5 6 7 8];  
>> matB = [4 5];  
>> matC = matA * matB  
Error using *  
Incorrect dimensions for matrix  
multiplication. Check that the number of  
columns in the first matrix matches the number  
of rows in the second matrix. To operate on  
each element of the matrix individually, use  
TIMES (.* ) for elementwise multiplication.
```

Related documentation

```
>> matC = matB * matA
```

```
matC =
```

Linear Algebra: Matrix Transpose

The transpose of an $m \times n$ matrix A is denoted by A^T , a $n \times m$ matrix where the rows and columns have been interchanged.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} 2 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 3 & 0 & 0 & 2 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$$A^T = \begin{bmatrix} a_{11} & a_{21} & a_{31} & a_{41} \\ a_{12} & a_{22} & a_{32} & a_{42} \\ a_{13} & a_{23} & a_{33} & a_{43} \\ a_{14} & a_{24} & a_{34} & a_{44} \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 & -1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 2 & 0 \end{bmatrix}$$

In MatLab, the transpose operation is performed using the ***transpose operator*** `'`.

```
>> a = [2,1,0,1; 1,0,1,1; 3,0,0,2;-1,1,1,0]
a =

     2     1     0     1
     1     0     1     1
     3     0     0     2
    -1     1     1     0

>> a_tran = a'
ans =

     2     1     3    -1
     1     0     0     1
     0     1     0     1
     1     1     2     0
```


Linear Algebra: Matrix Determinants

- The determinant of a matrix $|A|$ is formally defined as:

$$\det(A) = |A| = \sum_{j=1}^n a_{ij} (\text{cof}(a_{ij}))$$

$$\text{cof}(a_{ij}) = (-1)^{i+j} (\text{minor}(a_{ij}))$$

- Where minor a_{ij} is defined as the determinant of the submatrix obtained by deleting the i^{th} row and j^{th} column of matrix A.

- For a 2x2 matrix:

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$

MatLab provides the function **det()** that can find the determinant for us.

```
1 % 2x2 matrix
2 matA = [2,3; -4,6]
3 detA = det(matA)
4
5 % 3x3 matrix
6 matB = [3,-2,6; 1,3,4; 2,-1,2]
7 detB = det(matB)
```

```
matA =

     2     3
    -4     6
```

```
detA =
```

```
matB =

     3    -2     6
     1     3     4
     2    -1     2
```

```
detB =
```

Linear Algebra: Matrix Inverse

- Matrix B is the inverse of Matrix A if when the two matrices are multiplied the product is the identity matrix

$$AB = BA = I$$

- MatLab provides two options to calculate the inverse of a matrix:
 - the function `inv()` to obtain the inverse of a matrix.
 - The matrix exponent `sampMat^-1`

```
1 a = [2,1,4; 4,1,8; 2,-1,3]
2
3 aInv = inv(A)
4 % or...
5 aInv = a^-1
6
7
```

```
a =
     2     1     4
     4     1     8
     2    -1     3
```

```
aInv =
     5.5000    -3.5000     2.0000
     2.0000    -1.0000         0
    -3.0000     2.0000    -1.0000
```

```
aInv =
     5.5000    -3.5000     2.0000
     2.0000    -1.0000         0
    -3.0000     2.0000    -1.0000
```

```
i =
     1     0     0
     0     1     0
     0     0     1
```

⚠: A matrix will only have an inverse if it is non-singular (square, with a determinant that is not equal to zero).

Linear Algebra: Solving Linear Systems of Equations

- We can define a system of linear equations with n unknowns as

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$

- In matrix form, we can write

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

- Algebraically, we can multiply both sides of the matrix equation by the inverse of A and then simplify.

$$\begin{aligned}Ax &= b \\A^{-1}Ax &= A^{-1}b \\x &= A^{-1}b\end{aligned}$$

- In MatLab, the equation can be solved directly using MatLab's **inv()** function or through the use of the **matrix left divide operator **.

Example: Systems of Linear Equations

- Let's try to solve the following system of equations using MatLab:

$$\begin{aligned}2x + 5y &= 11 \\ 3x - 2y &= -12\end{aligned}$$

- Recasting this into matrix notation, we can write:

$$\begin{bmatrix} 2 & 5 \\ 3 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 11 \\ -12 \end{bmatrix}$$

```
1  % Set coefficient matrix (A)
2  A = [2,5; 3,-2];
3
4  % Vector of known outputs (b)
5  b = [11; -12];
6
7  % Solving the system of equations
8  x = inv(A) * b
9
10 % or ...
11 x2 = A\b
```

```
x =
    -2.0000
     3.0000
```

```
x2 =
    -2
     3
```

Relational and Logical Operators

Relational and logical operators can help us to ask questions about array elements.

- Relational operators allow us to ask questions about our numeric variables, to which MatLab will provide the answer of **true** (logical 1) or **false** (logical 0).

>	greater than	<	less than
>=	greater than or equal to	<=	less than or equal to
==	equality	~=	inequality

- Logical operators allow us to **combine the questions we** ask about our numeric variables, yielding a result of **true** (logical 1) or **false** (logical 0) for the combined condition.

: Making scalar decisions will become important when we get to if-statements.

Scalar Operators	
~	not
	or
&&	and

Element-Wise Array Operators	
~	not
	or
&	and

Comparing Arrays

- Array comparisons are always performed on an element-by-element basis, where the result is a **logical array** consisting of 1s (true) and 0s (false) that answers the comparison for each element.
- The arrays being compared must be the same size, or one must be a scalar.

```
vecA = [1 2 6 34 2 8];  
vecB = [5 1 6 80 2 7];  
  
test1 = vecA == vecB;  
test2 = vecB < vecA;  
test3 = vecA ~= 2;  
test4 = vecA >= vecB & vecB <= 2;  
test5 = vecA >= vecB && vecB <= 2;
```

Name	Value
vecA	[1 2 6 34 2 8]
vecB	[5 1 6 80 2 7]
test1	
test2	
test3	
test4	
test5	

Searching Arrays with Relational Indexing

When a logical array is of equivalent size to an array of interest, that logical vector can be used as an index vector for the array we are investigating.

We call this logical array a logical mask, in that it filters out all of the false values from the array.

```
1  vec1 = [25,-11,16,-33,21];
2  mat1 = [27,-32,41;30,0,-1];
3
4  vec2 = vec1>0;
5  mat2 = mat1 == 30;
6
7  vec3 = vec1(vec2);
8  vec4 = vec1;
9  vec4(vec2) = 0;
10
11 mat3 = mat1(mat1~=30);
12 mat4 = mat1;
13 mat4 = mat4(mat4<=0);
```

Name	Value
vec1	[25 -11 16 -33 21]
mat1	[27 -32 41; 30 0 -1]
vec2	[1 0 1 0 1]
mat2	0 0 0 1 0 0
vec3	[25 16 21]
vec4	[0 -11 0 -33 0]
mat3	[27; 30; -32; 0; -1]
mat4	

⚠: When your relational operation removes items from the matrix, the matrix is recast into a vector using linear indexing.

Searching Arrays with the `find()` Function

- MatLab provides a `find()` function that serves a similar function to the *logical mask*.
- `Find()` takes as an input a relational expression and searches the vector in question to return the address (indices) of the vector element that meets the search criterion.

```
1 midterm = [55, 99, 85, 40, 45, 93, 84, 39];  
2  
3 index1 = find(midterm == 93)  
4 index2 = find(midterm < 50)  
5 index3 = find(midterm == 100)
```

: Find does not return the element values, but rather returns the location (i.e. index) of the elements that satisfy the condition.

```
index1 =
```

```
index2 =
```

```
index3 =
```


Find() vs Relational Indexing

Given the same relational expression to search an array, `find()` and logical masks will convey similar information. However:

- a) Logical masks will generate an array of logical true/false values of equivalent size and shape to the original array
- b) MatLab's `find()` function will return a vector containing the indices where the specified conditions are true, which does not necessarily have the same number of elements as the original array.

```
1 midterm = [55,99,85,40,45,93,84,39];  
2  
3 %Search the vector  
4 mask = midterm > 80  
5 index = find(midterm > 80)  
6  
7 % how many grades are above 80%  
8 numMask = sum(mask)  
9 numFind = length(index)
```

```
mask =  
  
1×8 logical array  
  
0    1    1    0    0    1    1    0  
  
index =  
  
2     3     6     7  
  
numMask =  
  
4  
  
numFind =  
  
4
```

MATLAB Question: Environmental and Geological Engineering Applications

Part (a): Environmental Engineering — Water Quality After Treatment

The matrix **C** shows contaminant concentrations measured at three sampling locations.

Rows = Sampling Locations 1 to 3

Columns = Nitrate and Phosphate

```
C = [14.0 0.30;  
9.0 0.18;  
16.0 0.12];
```

```
removal = [25 40;  
20 30;  
35 20];
```

```
limit = [10 0.15;  
10 0.15;  
10 0.15];
```

The matrix **removal** gives the removal efficiency in percent.

Required MATLAB calculations

Use MATLAB to calculate:

```
Cfinal = C .* (1 - removal/100);
```

Then find:

1. The final concentration matrix **Cfinal**
2. The average final concentration of nitrate
3. The average final concentration of phosphate
4. How many final concentration values are still greater than the limit
5. The locations where nitrate is still greater than 10

Part (b): Geological Engineering — Groundwater Inflow from Fractures

Three fractures contribute to groundwater inflow into two boreholes.

The unknown fracture inflow rates are q_1 , q_2 , and q_3 .

The field equations are:

In MATLAB, use:

```
A = [2 1 1;  
1 3 2;  
1 0 1];
```

```
b = [13;  
18;  
7];
```

```
q = A\b;
```

$$2q_1 + q_2 + q_3 = 13$$

$$q_1 + 3q_2 + 2q_3 = 18$$

$$q_1 + q_3 = 7$$

Required MATLAB calculations

Find:

1. The determinant of **A**
2. The fracture inflow rates q_1 , q_2 , and q_3
3. The fracture number where inflow is greater than 4 L/min
4. The total inflow